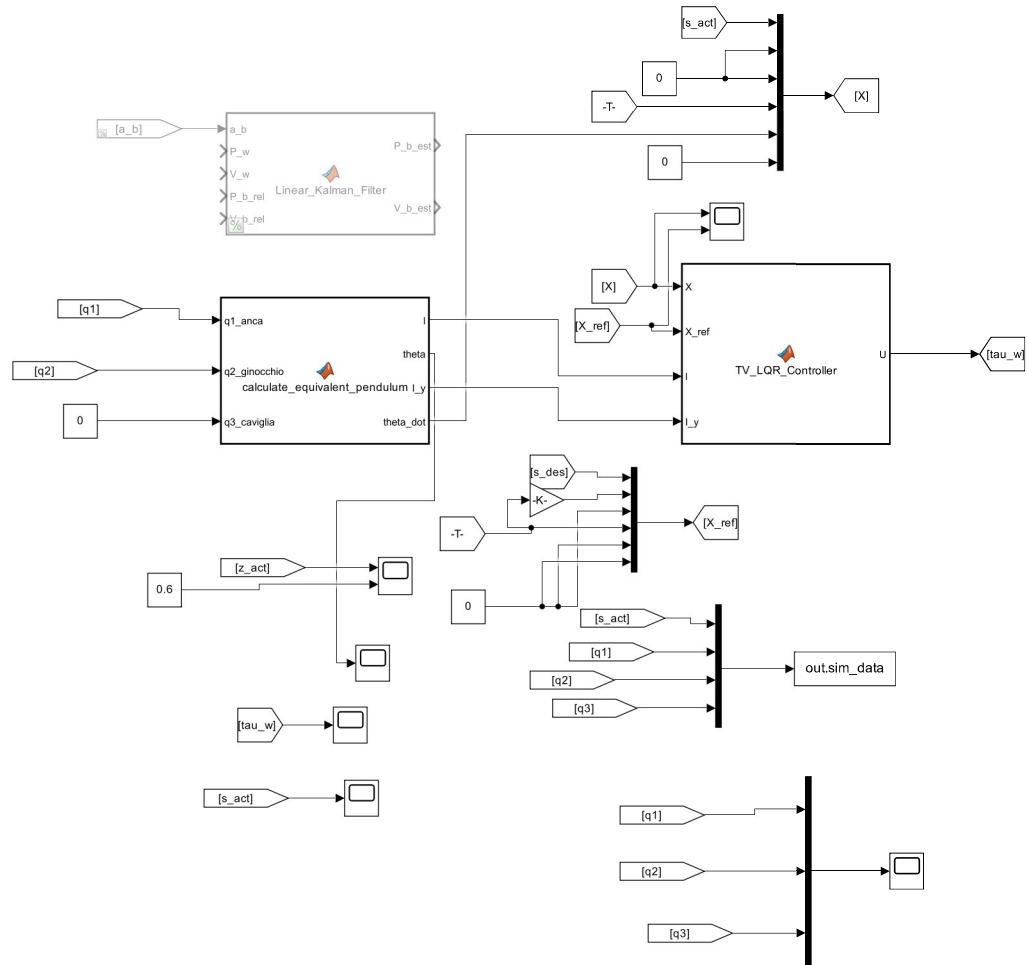
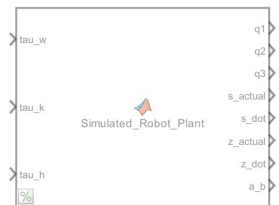
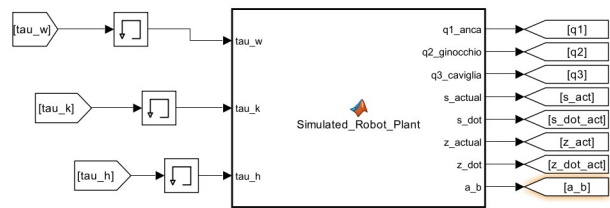
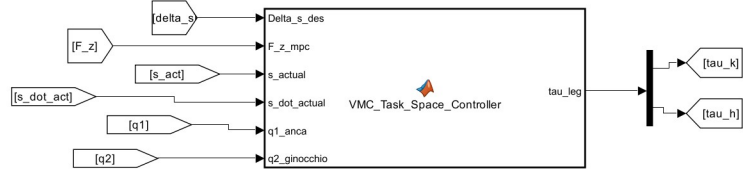
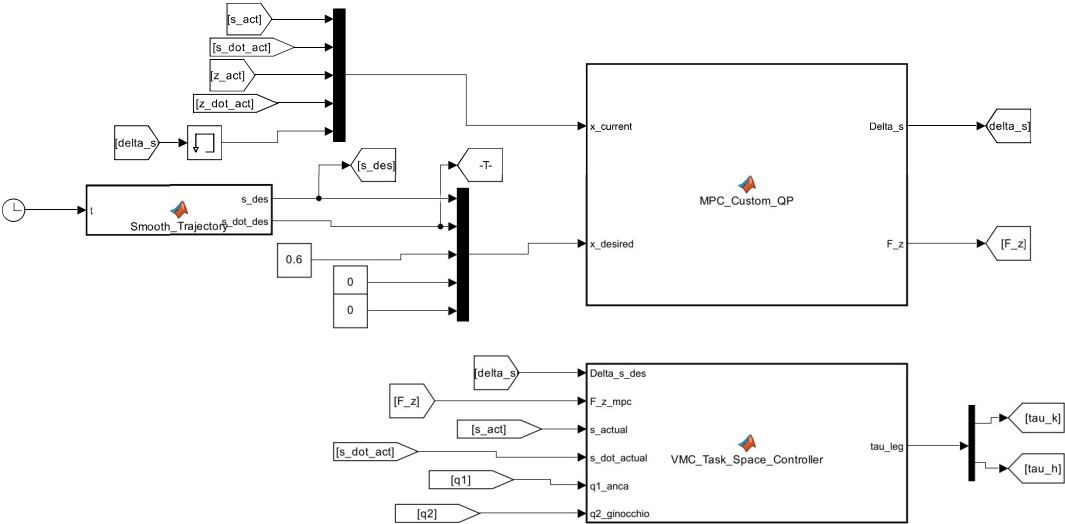


```

function [l, theta, I_y, theta_dot] = calculate_equivalent_pendulum(q1_anca, q2_ginocchio, q3_caviglia)
    persistent theta_prev
    if isempty(theta_prev), theta_prev = 0; end
    dt = 0.001;

    % Somma degli angoli della catena
    theta = q1_anca + q2_ginocchio + q3_caviglia;
    theta_dot = (theta - theta_prev) / dt;
    theta_prev = theta;

    l = 2 * 0.45 * cos(q2_ginocchio / 2);
    if l < 0.1, l = 0.1; end
    m_b = 73.0;
    I_y = (1/3) * m_b * l^2;
end

```

```

function U = TV_LQR_Controller(X, X_ref, l, I_y)
% X: Stato attuale [s; theta; phi; s_dot; theta_dot; phi_dot] (6x1)
% X_ref: Stato target desiderato [s_ref; theta_ref; phi_ref; 0; 0; 0] (6x1)
% l: Lunghezza attuale del pendolo equivalente [m]
% I_y: Inerzia attuale del corpo superiore [kg*m^2]

% Per usare la funzione lqr() di MATLAB all'interno di Simulink,
% dobbiamo dichiararla come estrinseca.
coder.extrinsic('lqr');

% 1. Richiamo Parametri Fisici
m_w = 3.5;      % Massa ruota
I_w = 0.1;      % Inerzia ruota
r_ = 0.127;     % Raggio ruota
d_ = 0.63;      % Distanza ruote
m_b = 73.0;     % Massa corpo superiore
I_z = 3.3;      % Inerzia torso asse z
g_ = 9.81;      % Gravità

% 2. Calcolo dei Coefficienti della Dinamica (Eq. 14)
% Denominatore comune per a1, a2, b1, b2
D = 2*I_w*(I_y + m_b*l^2) + (2*l^2*m_b*m_w + I_y*(m_b + 2*m_w)*r^2);

a1 = -(g * l^2 * m_b^2 * r^2) / D;
a2 = (g * l * m_b * (2*I_w + (m_b + 2*m_w)*r^2)) / D;

b1 = (r * (I_y + l*m_b*(l + r))) / D;
b2 = -(2*I_w + r*(l*m_b + (m_b + 2*m_w)*r)) / D;

% Coefficiente disaccoppiato per lo steering (imbardata)
b3 = (d * r) / (2*I_z*r^2 + d^2*(I_w + m_w*r^2));

% 3. Costruzione Matrici di Stato A(l) e Input B(l) (Eq. 12, Eq. 13)
A = zeros(6, 6);
A(1, 4) = 1;
A(2, 5) = 1;
A(3, 6) = 1;
A(4, 2) = a1;
A(5, 2) = a2;

B = zeros(6, 2);
B(4, 1) = b1; B(4, 2) = b1;
B(5, 1) = b2; B(5, 2) = b2;
B(6, 1) = b3; B(6, 2) = -b3;

% 4. Definizione Matrici di Peso Q e R (Tuning LQR)
% Questi valori determinano l'aggressività del controllore.
% Possono essere trasformati in input del blocco per un tuning rapido.
% 4. Definizione Matrici di Peso Q e R (Tuning Soft)
% Pesi drasticamente ridotti per evitare instabilità con il ritardo discreto
% Aumenta il peso sugli angoli (q1, q2) per evitare che saturino
Q = diag([10, 100, 100, 1, 10, 1]);
R = diag([1, 1]) * 1.0; % Penalizziamo maggiormente l'uso eccessivo dei motori

% 5. Risoluzione LQR e Calcolo Input
% Inizializza K per la Code Generation di Simulink
K = zeros(2, 6);

% Calcola la matrice dei guadagni ottimali K
K = lqr(A, B, Q, R);

```

```
% Errore di stato (tilde_X)
X_tilde = X - X_ref;

% Legge di controllo finale (Eq. 16): U = [tau_l; tau_r]
U = K * X_tilde;

end
```

```

function [P_b_est, V_b_est] = Linear_Kalman_Filter(a_b, P_w, V_w, P_b_rel, V_b_rel)
% INGRESSI:
% a_b: Accelerazione del torso misurata dall'IMU [2x1] (asse s, asse z)
% P_w: Posizione dell'asse ruote (odometria) [2x1]
% V_w: Velocità dell'asse ruote (odometria) [2x1]
% P_b_rel: Posizione del torso rispetto all'asse (cinematica diretta) [2x1]
% V_b_rel: Velocità del torso rispetto all'asse (giacobiano gambe) [2x1]

% Definisci dt (Tempo di campionamento, es. 1000 Hz dal paper)
dt = 0.001;

% Variabili di stato persistenti per mantenere la memoria tra i loop di Simulink
persistent X P
if isempty(X)
    X = zeros(4, 1); % Stato [Posizione_s; Posizione_z; Velocità_s; Velocità_z]
    P = eye(4);      % Matrice di covarianza dell'errore iniziale
end

% 1. Matrici del Modello (Eq. 20 e 21)
I2 = eye(2);
O2 = zeros(2, 2);

A = [I2, dt*I2;
     O2, I2];

B = [O2;
     dt*I2];

H = [I2, O2;
     O2, I2];

% 2. Matrici di Covarianza del Rumore (Da calibrare sul sistema reale)
Q = eye(4) * 0.01; % Rumore di processo (incertezza del modello)
R = eye(4) * 0.1;  % Rumore di misura (incertezza dei sensori)

% Nota dal paper: durante il salto, il rumore di osservazione (R)
% andrebbe aumentato perché l'encoder della ruota perde di affidabilità.

% --- FASE 1: PREDIZIONE (PREDICT) ---
% Previsione dello stato basata sull'accelerazione dell'IMU (Eq. 20)
X_pred = A * X + B * a_b;
P_pred = A * P * A' + Q;

% --- FASE 2: AGGIORNAMENTO (UPDATE) ---
% Costruzione della misurazione Z (Eq. 21)
% Z contiene la posizione assoluta (Ruota + Relativa) e velocità
Z = [P_w + P_b_rel;
     V_w + V_b_rel];

% Calcolo del guadagno di Kalman
S = H * P_pred * H' + R;
K_kalman = P_pred * H' / S;

% Aggiornamento dello stato stimato e della covarianza
X = X_pred + K_kalman * (Z - H * X_pred);
P = (eye(4) - K_kalman * H) * P_pred;

% USCITE: Estrazione di Posizione e Velocità stimate
P_b_est = X(1:2);
V_b_est = X(3:4);
end

```

end


```

function [Delta_s, F_z] = MPC_Custom_QP(x_current, x_desired)
% quadprog non è supportato nativamente per la code generation C
coder.extrinsic('quadprog');
coder.extrinsic('optimoptions');

%% 1. Parametri e Discretizzazione
m_b = 73.0;
g = 9.81;

h = x_current(3);
if h < 0.1
    h = 0.6;
end

dt = 0.01;
N = 10;

A_c = zeros(5,5);
A_c(1,2) = 1;
A_c(3,4) = 1;
A_c(2,5) = g / h;

B_c = zeros(5,2);
B_c(4,2) = 1 / m_b;
B_c(5,1) = 1;

A_d = eye(5) + A_c * dt;
B_d = B_c * dt;

%% 2. Pesi della Funzione di Costo
S = diag([100, 10, 500, 50, 1]);
W = diag([1, 0.01]);

%% 3. Condensazione (Srotolamento)
nx = 5; nu = 2;
P_x = zeros(nx*N, nx);
P_u = zeros(nx*N, nu*N);

A_power = eye(nx);
for i = 1:N
    A_power = A_power * A_d;
    P_x((i-1)*nx+1 : i*nx, :) = A_power;
    for j = 1:nu
        P_u((i-1)*nx+1 : i*nx, (j-1)*nu+1 : j*nu) = (A_d^(i-j)) * B_d;
    end
end

S_bar = kron(eye(N), S);
W_bar = kron(eye(N), W);

H = 2 * (P_u' * S_bar * P_u + W_bar);
H = (H + H') / 2; % Garantisce simmetria perfetta

X_ref = repmat(x_desired, N, 1);
f = 2 * P_u' * S_bar * (P_x * x_current - X_ref);

%% 4. Vincoli Fisici
F_min = -1500; % Permette di frenare!
F_max = 1500;

LB = zeros(nu*N, 1);

```

```

UB = zeros(nu*N, 1);

for i = 1:N
    LB((i-1)*nu+1) = -2.0;
    UB((i-1)*nu+1) = 2.0;

    LB((i-1)*nu+2) = F_min;
    UB((i-1)*nu+2) = F_max;
end

%% 5. Risoluzione QP (CORRETTA)
% Pre-allocazione per Simulink Coder
U_opt_calc = zeros(nu*N, 1);
fval = 0.0;
exitflag = 0.0;

% Algoritmo robusto di default
options = optimoptions('quadprog', 'Display', 'off');

% LA CORREZIONE: Aggiunto "fval" per non sballare "exitflag"!
[U_opt_calc, fval, exitflag] = quadprog(H, f, [], [], [], [], LB, UB, [], options);

if exitflag > 0
    U_opt = U_opt_calc;
else
    % Se fallisce davvero, eroga 0 extra, ma il VMC ci terrà in piedi col peso base
    U_opt = zeros(nu*N, 1);
end

%% 6. Estrazione Input Ottimo
Delta_s_dot_opt = U_opt(1);
F_z = U_opt(2); % FINALMENTE SARA' DIVERSO DA ZERO!

Delta_s = x_current(5) + Delta_s_dot_opt * dt;
end

```

```

function tau_leg = VMC_Task_Space_Controller(Delta_s_des, F_z_mpc, s_actual, s_dot_actual, q1_anca, q2_ginocchio)
    tau_leg = zeros(2,1);
    l = 0.45; m_tot = 73.0; g = 9.81;

    % Jacobiano lineare per il ginocchio
    J_z_knee = -l * sin(q2_ginocchio / 2);
    if abs(J_z_knee) < 0.05, J_z_knee = -0.05 * sign(q2_ginocchio + 1e-6); end

    F_z_totale = F_z_mpc + (m_tot * g);

    % Il ginocchio assorbe la spinta. L'anca sta a 0 (passiva)
    tau_k = J_z_knee * F_z_totale;
    tau_h = 0;

    tau_leg = [tau_k; tau_h];
end

```

```
function [q1, q2, q3, s_actual, s_dot, z_actual, z_dot, a_b] = Simulated_Robot_Plant(tau_w, tau_k, tau_h)
```

```
% --- 0. PRE-ALLOCAZIONE PER IL CODER SIMULINK ---  
% Diciamo esplicitamente a Simulink le dimensioni e il tipo (double) delle uscite  
q1 = 0.0;  
q2 = 0.0;  
q3 = 0.0;  
s_actual = 0.0;  
s_dot = 0.0;  
z_actual = 0.0;  
z_dot = 0.0;  
a_b = zeros(2,1);
```

```
% --- 1. INIZIALIZZAZIONE FISICA ---  
persistent X_robot  
if isempty(X_robot)  
    X_robot = [0; 0; 0.6; 0]; % [s, s_dot, z, z_dot]  
end
```

```
dt = 0.001;  
m_b = 73;  
g = 9.81;  
r = 0.127; % Raggio ruota
```

```
%% 2. FISICA ASSE Z (Verticale)  
J21_approx = 0.33;  
F_z_applied = -(tau_k + tau_h) / J21_approx;
```

```
% Limite di sicurezza fisico per evitare esplosioni numeriche  
if F_z_applied > 2000, F_z_applied = 2000; end  
if F_z_applied < 0, F_z_applied = 0; end
```

```
z_ddot = (F_z_applied / m_b) - g;
```

```
%% 3. FISICA ASSE S (Orizzontale)  
F_s_applied = (tau_w(1) + tau_w(2)) / r;  
s_ddot = F_s_applied / m_b;
```

```
%% 4. INTEGRAZIONE DI EULERO  
X_robot(4) = X_robot(4) + z_ddot * dt;  
X_robot(3) = X_robot(3) + X_robot(4) * dt;
```

```
X_robot(2) = X_robot(2) + s_ddot * dt;  
X_robot(1) = X_robot(1) + X_robot(2) * dt;
```

```
% Se il robot "cade a terra", il pavimento lo ferma (z = 0.1)  
if X_robot(3) < 0.1  
    X_robot(3) = 0.1;  
    X_robot(4) = 0;  
end
```

```
z_actual = X_robot(3);  
z_dot = X_robot(4);  
s_actual = X_robot(1);  
s_dot = X_robot(2);
```

```
%% 5. CINEMATICA INVERSA (Corretta per scendere dritti!)
```

```
l = 0.45;  
ratio = z_actual / (2 * l);
```

```
if ratio > 0.999
```

```
        ratio = 0.999;
elseif ratio < -0.999
    ratio = -0.999;
end

% Manteniamo il bacino centrato sopra la ruota
q1 = 2 * acos(ratio);
q2 = q1 / 2;

q3 = 0.0; % Torso dritto
%% 6. IMU
a_b = [s_ddot; z_ddot + g];
end
```

```

function [s_des, s_dot_des] = Smooth_Trajectory(t)
    % PRE-ALLOCAZIONE PER IL CODER
    s_des = 0.0;
    s_dot_des = 0.0;

    % PARAMETRI DELLA TRAIETTORIA
    t_start = 2.0;          % Il robot inizia a muoversi a 2 secondi
    t_end = 5.0;            % Arriva a destinazione a 5 secondi (3 secondi di viaggio)
    s_final = 1.0;          % Destinazione orizzontale (1 metro)

    % LOGICA DEL MOVIMENTO
    if t <= t_start
        % Prima di partire, resta fermo
        s_des = 0.0;
        s_dot_des = 0.0;

    elseif t >= t_end
        % Arrivato a destinazione, fermati
        s_des = s_final;
        s_dot_des = 0.0;

    else
        % FASE DI MOTO: Polinomio Cubico (Minimum Jerk Trajectory)
        % Normalizziamo il tempo tra 0 e 1 per la durata del viaggio
        tau = (t - t_start) / (t_end - t_start);

        % Posizione (Profilo a "S" morbida)
        s_des = s_final * (3*tau^2 - 2*tau^3);

        % Velocità (Derivata della posizione, forma a campana)
        s_dot_des = s_final * (6*tau - 6*tau^2) / (t_end - t_start);
    end
end
end

```

```

function [q1_anca, q2_ginocchio, q3_caviglia, s_actual, s_dot, z_actual, z_dot, a_b] = Simulated_Robot_Plant(tau_w, tau_k, tau_h)
q1_anca = 0.0; q2_ginocchio = 0.0; q3_caviglia = 0.0;
s_actual = 0.0; s_dot = 0.0; z_actual = 0.0; z_dot = 0.0;
a_b = zeros(2,1);

persistent X_z X_wip
if isempty(X_z)
    X_z = [0; 0];
    X_wip = [0; 0; 0; 0];
end
dt = 0.001; g = 9.81; m_b = 73.0; r = 0.127; m_w = 3.5; I_w = 0.1; l = 0.45;

%% 1. FISICA VERTICALE (Z)
ratio = X_z(1) / (2*l);
if ratio > 0.999, ratio = 0.999; elseif ratio < -0.999, ratio = -0.999; end
q2_current = 2 * acos(ratio);

J_z_knee = -l * sin(q2_current / 2);
if abs(J_z_knee) < 0.05, J_z_knee = -0.05 * sign(q2_current + 1e-6); end

F_z_applied = tau_k / J_z_knee;
z_ddot = (F_z_applied / m_b) - g;

X_z(2) = X_z(2) + z_ddot * dt;
X_z(1) = X_z(1) + X_z(2) * dt;

% Blocchi fisici estremi per evitare decolli o sprofondamenti
if X_z(1) < 0.2, X_z(1) = 0.2; X_z(2) = 0; end
if X_z(1) > 0.9, X_z(1) = 0.9; X_z(2) = 0; end
z_act = X_z(1);

%% 2. FISICA DEL PENDOLO Orizzontale
l_pend = z_act;
if l_pend > 0.9, l_pend = 0.9; end
if l_pend < 0.1, l_pend = 0.1; end

I_y = (1/3) * m_b * l_pend^2;

D = 2*I_w*(I_y + m_b*l_pend^2) + (2*l_pend^2*m_b*m_w + I_y*(m_b + 2*m_w)*r^2);
a1 = -(g * l_pend^2 * m_b^2 * r^2) / D;
a2 = (g * l_pend * m_b * (2*I_w + (m_b + 2*m_w)*r^2)) / D;
b1 = (r * (I_y + l_pend*m_b*(l_pend + r))) / D;
b2 = -(2*I_w + r*(l_pend*m_b + (m_b + 2*m_w)*r)) / D;

A_wip = [0 0 1 0; 0 0 0 1; 0 a1 0 0; 0 a2 0 0];
B_wip = [0 0; 0 0; b1 b1; b2 b2];

% LA CORREZIONE VITALE: Diamo ai motori la forza di bilanciare 73 kg!
tau_w_lim = tau_w;
limite = 1000; % Aumentato da 15 a 1000 Nm
if tau_w_lim(1) > limite, tau_w_lim(1) = limite; end
if tau_w_lim(1) < -limite, tau_w_lim(1) = -limite; end
if tau_w_lim(2) > limite, tau_w_lim(2) = limite; end
if tau_w_lim(2) < -limite, tau_w_lim(2) = -limite; end

X_wip_dot = A_wip * X_wip + B_wip * tau_w_lim;
X_wip = X_wip + X_wip_dot * dt;
theta_vero = X_wip(2);

%% 3. ASSEGNAZIONE GIUNTI (La tua convenzione perfetta)
ratio_new = z_act / (2*l);

```

```
if ratio_new > 0.999, ratio_new = 0.999; elseif ratio_new < -0.999, ratio_new = -0.999; end  
q2_ginocchio = 2 * acos(ratio_new);
```

```
alpha = q2_ginocchio / 2;  
q3_caviglia = -alpha + theta_vero;  
q1_anca = -alpha;
```

```
%% 4. ASSEGNAZIONE USCITE
```

```
s_actual = X_wip(1);  
s_dot = X_wip(3);  
z_actual = z_act;  
z_dot = X_z(2);  
a_b = [X_wip_dot(3); z_ddot + g];
```

```
end
```